

Análisis de la expresión génica de levadura (*Saccharomyces cerevisiae*) bajo diferentes condiciones de crecimiento

Problema biológico

Estudiar el transcriptoma e investigar los cambios en los niveles de expresión génica de la cepa [CEN.PK 113-7D](#) bajo dos condiciones metabólicas diferentes:

1. Metabolismo respiro-fermentativo (*batch*)
2. Totalmente respiratorio (*quimiostato*)

En el primer caso, el crecimiento es un proceso discontinuo en el que la levadura se cultiva en un medio limitado de nutrientes y oxígeno. La levadura utiliza primero el oxígeno para producir energía a través de la respiración celular y, una vez agotado el oxígeno, comienza a producir energía a través de la fermentación. En este proceso, la levadura consume azúcares para producir dióxido de carbono y alcohol.

Por otro lado, el crecimiento de levadura en un metabolismo completamente respiratorio (quimiostato) es un proceso continuo en el que la levadura se cultiva en un medio con una tasa de flujo constante de nutrientes y oxígeno. En este proceso, la levadura utiliza exclusivamente el oxígeno presente para producir energía a través de la respiración celular, lo que resulta en un crecimiento más lento pero una mayor producción de biomasa. Este metabolismo es útil para producir levadura para su uso en la fabricación de pan o para la producción de proteínas recombinantes en la industria farmacéutica.

Los datos de RNA-Seq están disponibles en la base de datos SRA de NCBI, bajo el número de acceso SRS307298. En total hay **dos muestras, dos tratamientos con tres réplicas biológicas**. Los datos son de tipo *paired-end*. Se nos brindaron sólo las lecturas que alinean contra el cromosoma I de levadura.

Muestra	Condición	Experimento	Réplica	SRA Accession	Protocolo preparación biblioteca	Read length	Stranded?
Quimiostato_1	Control	RNA-seq	1	SRR453569	Captura por polyA, paired end		No
Quimiostato_2	Control	RNA-seq	2	SRR453570	Captura por polyA, paired end		No

Quimiostato_3	Control	RNA-seq	3	SRR453571	Captura por polyA, paired end		No
Batch_1	Tratamiento	RNA-seq	1	SRR453566	Captura por polyA, paired end		No
Batch_2	Tratamiento	RNA-seq	2	SRR453567	Captura por polyA, paired end		No
Batch_3	Tratamiento	RNA-seq	3	SRR453568	Captura por polyA, paired end		No

Control de calidad y estadísticas de las secuencias

Número de lecturas y largo

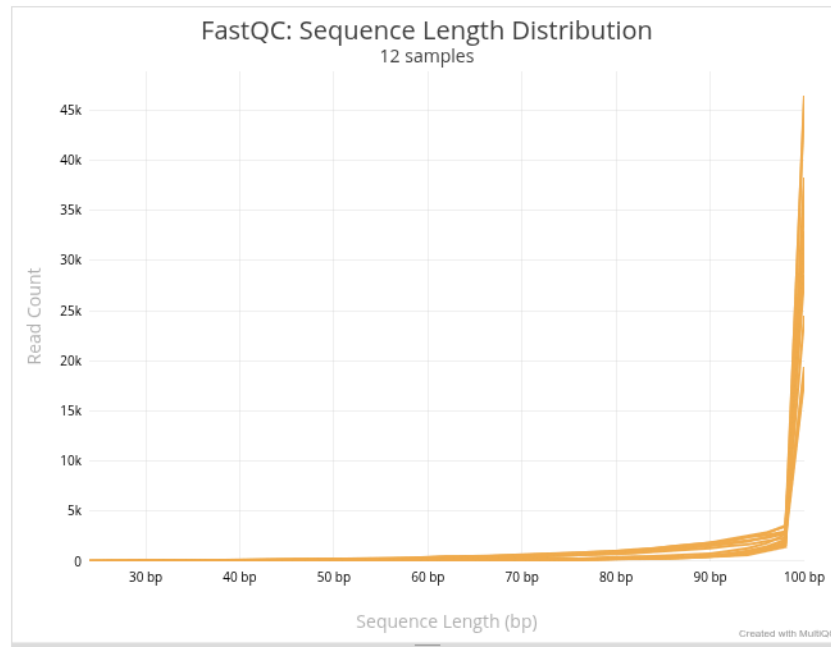
```
for fastq in *fastq
do echo $fastq
cat $fastq | awk 'BEGIN{i=0}{i++;}END{print i/4}'
done
```

Divido por 4 porque el FASTQ tiene 4 líneas

Archivo	Lecturas
batch1_1.fastq	44050
batch1_2.fastq	44050
batch2_1.fastq	59900
batch2_2.fastq	59900
batch3_1.fastq	46775
batch3_2.fastq	46775
quimiostato1_1.fastq	38325
quimiostato1_2.fastq	38325
quimiostato2_1.fastq	47162
quimiostato2_2.fastq	47162

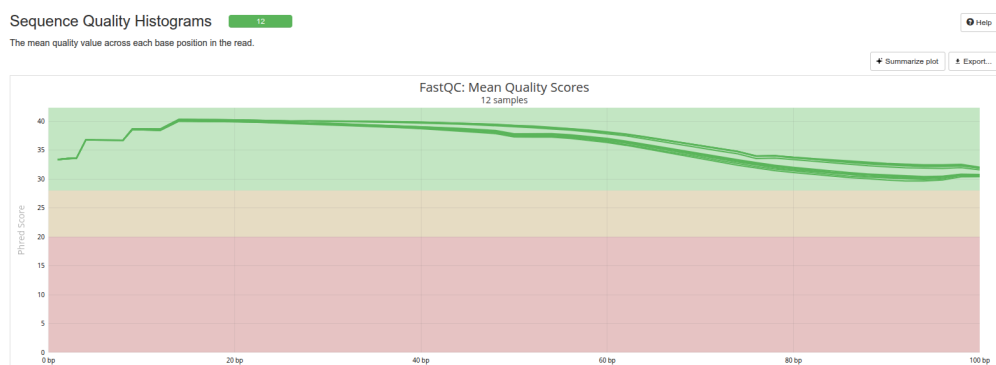
quimiostato3_1.fastq	56865
quimiostato3_2.fastq	56865

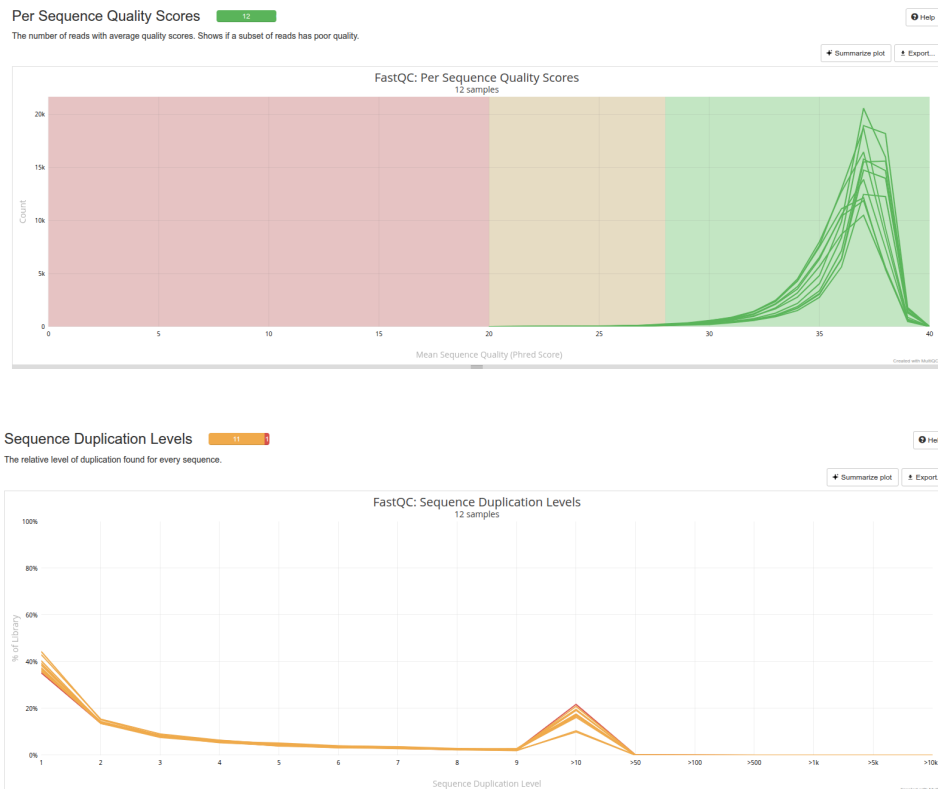
La mayoría tiene entre 38k y 56k de lecturas



Las lecturas ya están procesadas, entonces tengo de diversos largos. La mayoría tienen 101, seguro este fue el de las lecturas originales.

Control de calidad





La calidad de las secuencias es excelente, probablemente porque como noté están preprocesadas. Además, veo cierto nivel de duplicación que en RNA-seq es de esperarse, estoy cuantificando.

Integración con Snakemake

Simplemente cree dos reglas, una para correr FastQC con cada archivo de reads. La segunda lo que hace es, una vez que estén presentes todos los archivos reporte de fastqc, integrarlos con multiqc.

```
rule fastqc:
    input:
        os.path.join(SAMPLES_DIR, "{sample}.fastq")
    output:
        html = "results/quality_control/fastqc/{sample}_fastqc.html",
        zip = "results/quality_control/fastqc/{sample}_fastqc.zip"
    shell:
        "fastqc {input} --outdir results/quality_control/fastqc/ -q"

rule multiqc:
    input:
        expand("results/quality_control/fastqc/{sample}_fastqc.html", sample=SAMPLES)
    output:
        "results/quality_control/multiqc/multiqc_report.html"
    shell:
        "multiqc results/quality_control/fastqc/ --outdir results/quality_control/multiqc"
```

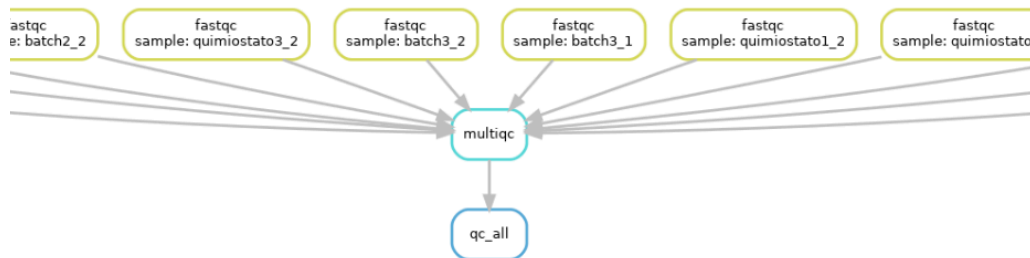
Por último, definí una regla muy simple para correr todo este grupo de reglas por separado cuando se quiera:

```
rule qc_all:
    input:
        "results/quality_control/multiqc/multiqc_report.html",
```

Entonces si interesa correr el control de calidad simplemente requeriría:

```
snakemake --cores all qc_all
```

Este flujo quedaría de la siguiente forma con la relación de inputs y outputs que definí para el control de calidad, incluyó una versión recortada para una mejor visualización, tendremos una ejecución de fastqc para cada archivo de read :



De ejecutar este flujo se obtienen los siguientes resultados:

```
[4.0K] results/quality_control//
├── [4.0K] fastqc/
│   ├── [228K] batch1_1_fastqc.html
│   ├── [264K] batch1_1_fastqc.zip
│   ├── [229K] batch1_2_fastqc.html
│   ├── [267K] batch1_2_fastqc.zip
│   ├── [227K] batch2_1_fastqc.html
│   ├── [266K] batch2_1_fastqc.zip
│   ├── [226K] batch2_2_fastqc.html
│   ├── [262K] batch2_2_fastqc.zip
│   ├── [229K] batch3_1_fastqc.html
│   ├── [266K] batch3_1_fastqc.zip
│   ├── [229K] batch3_2_fastqc.html
│   ├── [267K] batch3_2_fastqc.zip
│   ├── [228K] quimiostato1_1_fastqc.html
│   ├── [263K] quimiostato1_1_fastqc.zip
│   ├── [228K] quimiostato1_2_fastqc.html
│   ├── [266K] quimiostato1_2_fastqc.zip
│   ├── [230K] quimiostato2_1_fastqc.html
│   ├── [266K] quimiostato2_1_fastqc.zip
│   ├── [231K] quimiostato2_2_fastqc.html
│   ├── [269K] quimiostato2_2_fastqc.zip
│   ├── [229K] quimiostato3_1_fastqc.html
│   ├── [266K] quimiostato3_1_fastqc.zip
│   ├── [228K] quimiostato3_2_fastqc.html
│   └── [265K] quimiostato3_2_fastqc.zip
├── [4.0K] multiqc/
│   ├── [4.0K] multiqc_data/
│   └── [4.9M] multiqc_report.html
```

Como se puede observar en la figura se crea una carpeta de control de calidad en resultados, dentro una para cada herramienta y allí podemos encontrar toda la información relevante de la ejecución de estos programas.

Alineamiento de las lecturas contra el genoma de referencia

Tenemos el genoma de referencia que corresponde solo al cromosoma I de *S. cerevisiae* (ensamblado sacCer2) y el archivo de anotación (gtf) del genoma completo.

Decidí llevar a cabo el alineamiento con STAR, porque es splice aware y rápido. Además tiene facilidades para contar los genes, ordenar los archivos BAM, etc.

Creación del Índice con STAR

Para indexar los archivos tal como lo necesita STAR, usé el modo de generación que este mismo alineador tiene. Lo integré en una regla de snakemake:

```
rule make_star_index:
    input:
        expand("data/reads/{sample}.fastq", sample=SAMPLES),
        genome_reference = "data/reference/sacCer_chrI.fa",
        gtf_reference = "data/reference/sacCer_genes.gtf"
    output:
        touch("results/mapping/index_STAR/.index_complete") # Archivo bandera
    params:
        genomeDir="results/mapping/index_STAR",
        genomeFastaFiles=input.genome_reference,
        sjdbGTFfile=input.gtf_reference,
        sjdbOverhang=100,
        genomeSAindexNbases=lambda wildcards, input: min(14, math.log2(sum(len(record) for record in Bio.SeqIO.parse(input.genome_reference, "fasta")))/2 - 1),
        outFileNamePrefix="sacCer_chr1",
    threads: 4
    shell:
        """
        STAR --runMode genomeGenerate --genomeDir {params.genomeDir} \
            --genomeFastaFiles {params.genomeFastaFiles} \
            --sjdbGTFfile {params.sjdbGTFfile} \
            --sjdbOverhang {params.sjdbOverhang} \
            --genomeSAindexNbases {params.genomeSAindexNbases} \
            --outFileNamePrefix {params.outFileNamePrefix} \
            --runThreadN {threads}
        """
```

Toma como entrada todos los fastq, el genoma de referencia y el GTF. SAMPLES está definido en el .smk principal.

Después, con params de snakemake definí los parámetros para la ejecución de la indexación. Para más facilidad le dí los mismos nombres que ya llevan de por sí. Estos son:

- genomeDir: dónde queremos que se guarden los índices. Definí una carpeta dentro de results para todos los resultados de mapping.
- genomeFastaFiles: la ruta al genoma de referencia, se pasa como input a la regla.
- sjdbGTFfile: la ruta al gtf de referencia, también viene del input.
- sjdbOverhang:

- genomeSAindexNbases: es el mínimo entre 14 y el número de bases del genoma de referencia sobre 2, menos 1. En este caso use biopython para el largo del genoma, para no hardcodearlo, sino que se calcula dinámicamente en tiempo de ejecución.
- outFileNamePrefix: Es el prefijo que quiero que use para los archivos de salida

Puse como salida de esta regla un archivo bandera, que si no hay problemas durante la ejecución de la indexación, se crea para luego ser tomado como referencia para lanzar la ejecución del mapeo propiamente dicho.

Mapeo con STAR

```
rule align_STAR:
    input:
        fastq_1 = "data/reads/{replic}_1.fastq",
        fastq_2 = "data/reads/{replic}_2.fastq",
        index_flag = "results/mapping/index_STAR/.index_complete"
    output:
        bam = "results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam",
        counts = "results/mapping/alignments_STAR/{replic}/{replic}ReadsPerGene.out.tab",
        log = "results/mapping/alignments_STAR/{replic}/{replic}Log.final.out"
    params:
        prefix = "results/mapping/alignments_STAR/{replic}/{replic}",
        genomeDir = "results/mapping/index_STAR"
    threads: 4
    shell:
        """
        STAR --genomeDir {params.genomeDir} \
            --readFilesIn {input.fastq_1},{input.fastq_2} \
            --outSAMtype BAM SortedByCoordinate \
            --quantMode GeneCounts \
            --outFileNamePrefix {params.prefix} \
            --runThreadN {threads}
        """
```

Siguiendo la lógica de desarrollo en Snakemake, cree una regla para el mapeo con STAR. Tomo como entrada, cuando estén estos archivos se va a lanzar, las lecturas (los pares por réplica).

Como salida, defino donde quiero que se guarde el archivo bam, con la lógica de carpetas para que dentro de los alineamientos queden en carpetas por réplica (batch1, batch2, etc)

Luego defino los parámetros:

- genomeDir: el directorio donde están los index. Es el que use en el indexado.
- readFilesIn: paso las rutas a las reads, separadas por coma.
- outSAMtype: el tipo de archivo SAM que queremos, binario en nuestro caso, ordenado por coordenadas.

- quantMode: Como queremos cuantificar, en mi caso con GeneCounts, quiero que se cuenten los genes.
- outFileNamePrefix: el prefijo que quiero que se use, debe ser el mismo que puse que va a salir el bam, sino me va a dar un error porque falta el output.
- runThreadN: el número de threads que quiero que se usen, yo los setee con threads de snakemake en 4.

De correr estos comandos obtengo los archivos de salida de STAR esperados y que vimos en practica, como ejemplo para batch1:

```

[4.0K] mapping/
├── [4.0K] alignments_STAR/
│   └── [4.0K] batch1/
│       ├── [5.2M] batch1Aligned.sortedByCoord.out.bam
│       ├── [1.6K] batch1Aligned.sortedByCoord.out.bam.bai
│       ├── [ 17K] batch1Coverage.bw
│       ├── [1.9K] batch1Log.final.out
│       ├── [6.2K] batch1Log.out
│       ├── [ 246] batch1Log.progress.out
│       ├── [2.4K] batch1ReadsPerGene.out.tab
│       └── [ 223] batch1SJ.out.tab

```

Ejecución de Qualimap

Teniendo ya los alineamientos, procedí a ejecutar qualimap para hacer un análisis de la calidad del alineamiento.

```

rule quality_control_mapping:
    input:
        alignment="results/mapping/alignments_STAR/{sample}/{sample}Aligned.sortedByCoord.out.bam",
        gtf="data/reference/sacCer_genes.gtf",
        gene_counts="results/mapping/alignments_STAR/{sample}/{sample}ReadsPerGene.out.tab"
    output:
        "results/qc_qualimap/{sample}/qualimapReport.html"
    params:
        outdir="results/qc_qualimap/{sample}",

    shell:
        """
        protocol=$(grep -v "N_" {input.gene_counts} | \
            awk '{{unst+=$2;forw+=$3;rev+=$4}} \
            END{{if (forw > 2*rev) print "strand-specific-forward"; \
            else if (rev > 2*forw) print "strand-specific-reverse"; \
            else print "non-strand-specific"}}}')

        qualimap rnaseq -bam {input.alignment} \
            -gtf {input.gtf} \
            -outdir {params.outdir} \
            -p $protocol \
            -pe

        """

```

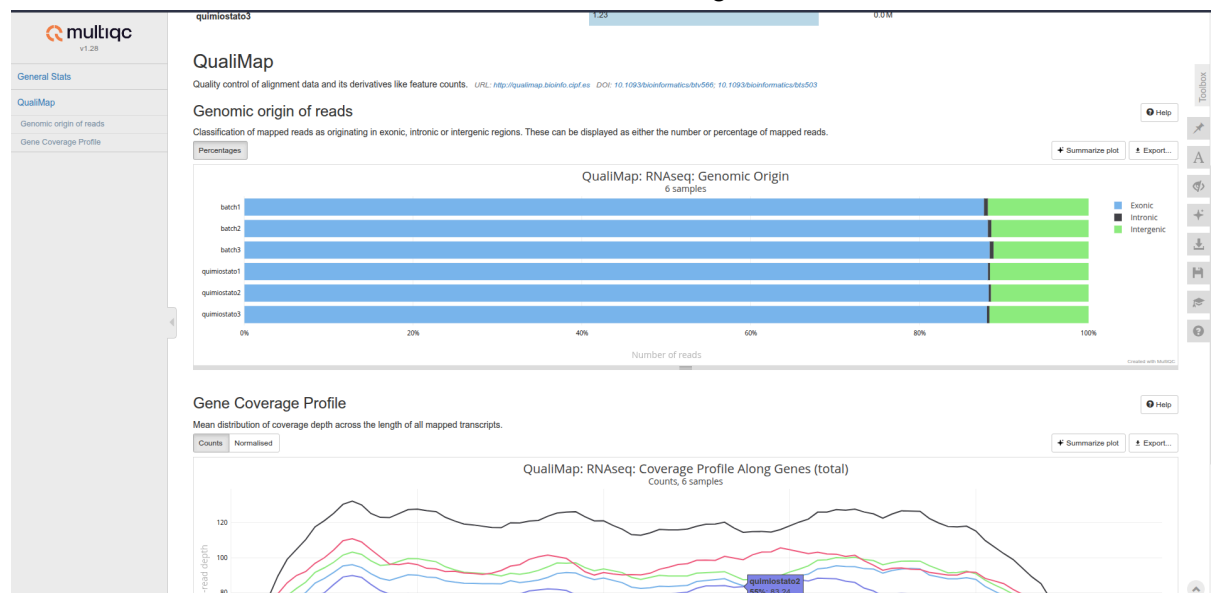
Definí una regla general para ejecutar qualimap para un alineamiento, luego lo puedo aplicar a cada muestra.

Simplemente toma como input el genecounts, el alineamiento en BAM y el gtf. Luego define la ruta al reporte de qualimap.

En primer lugar cree un pequeño comando para ver si es stranded o unstranded. Básicamente si tiene más del doble de presencia de una hebra lo define como stranded y de qué hebra.

Teniendo el tipo de protocolo ejecutó Qualimap para rnaseq.

Tras inspeccionar todos los reportes de Qualimap por separados, cree un reporte con MultiQC para integrar todos los Qualimap y pude ver que todos los alineamiento tenían buenas calidades, alineando todos cerca del 80% a regiones exónicas.



Visualización del alineamiento

Para visualizar el alineamiento decidí usar IGV, porque ví que se puede importar como JavaScript directamente en un HTML de forma muy fácil.

Primero lo que hice fue indexar el archivo BAM y calcular el coverage en un archivo de BigWig con samtools y deeptools respectivamente.

```

rule indexing_bam:
    input:
        bam_file="results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam"
    output:
        "results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam.bai"
    shell:
        """
        samtools index {input.bam_file}
        """

rule calculate_coverage:
    input:
        bam_file="results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam",
        bai_file="results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam.bai"
    output:
        coverage_file="results/mapping/alignments_STAR/{replic}/{replic}Coverage.bw"
    shell:
        """
        bamCoverage -b {input.bam_file} -o {output.coverage_file}
        """

```

Ambas reglas las define de forma general para luego poder ejecutarla para todas las réplicas.

Llegado a este punto cree dos templates, uno para la config en un JS y otro para el HTML en sí del reporte que quiero generar:

```

<!DOCTYPE html>
<html>
<head>
    <script src="https://cdn.jsdelivr.net/npm/igv@2.12.3/dist/igv.min.js"></script>
    <script src="igv_config.js"></script>
    <style>#igv-container { height: 90vh; margin: 20px; }</style>
</head>
<body>
    <div id="igv-container"></div>
    <script>
        document.addEventListener("DOMContentLoaded", () => {
            const igvDiv = document.getElementById("igv-container");
            igv.createBrowser(igvDiv, igvConfig);
        });
    </script>
</body>
</html>

```

```
// Marcador para ser reemplazado
const igvConfig = {{IGV_CONFIG}};
```

Ambos templates los almacené en la carpeta data/templates. En el HTML simplemente incluyo desde internet el igv.js, podría descargarse para que esté disponible por más que no haya conexión a internet pero no creo que sea el caso. Luego creo un container y creo el browser pasándole el contenedor y la configuración que quiero usar.

La configuración que hay que hacer es muy simple, como ejemplo cito el de su github:

```
{
  genome: "hg38",
  locus: "chr8:127,736,588-127,739,371",
  tracks: [
    {
      "name": "HG00103",
      "url":
        "https://s3.amazonaws.com/1000genomes/data/HG00103/alignment/HG00103.alt_bwamem_
        GRCh38DH.20150718.GBR.low_coverage.cram",
      "indexURL":
        "https://s3.amazonaws.com/1000genomes/data/HG00103/alignment/HG00103.alt_bwamem_
        GRCh38DH.20150718.GBR.low_coverage.cram.crai",
      "format": "cram"
    }
  ]
};
```

En mi caso en tracks voy a tener un Bigwig por réplica.

Volviendo al Snakemake, simplemente defino un json con las rutas a mis archivos, tanto genoma, gtf como bigwig.

```

rule generate_igv_tracks:
    input:
        bws = expand("results/mapping/alignments_STAR/{replic}/{replic}Coverage.bw", replic=REPLICS),
        bams = expand("results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam", replic=REPLICS),
        bais = expand("results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam.bai", replic=REPLICS),
        fasta_reference = "data/reference/sacCer_Chrl.fa",
        fai_reference = "data/reference/sacCer_Chrl.fa.fai",
        gtf_reference = "data/reference/sacCer_genes.gtf"
    output:
        "results/igv_visualization/igv_tracks.json"
    run:
        import json

        tracks_config = {
            "genome": {
                "fastaURL": input.fasta_reference,
                "indexURL": input.fai_reference,
                "gtfURL": input.gtf_reference
            },
            "tracks": []
        }

        for replic in REPLICS:
            tracks_config["tracks"].append({
                "name": replic,
                "bw": f"results/mapping/alignments_STAR/{replic}/{replic}Coverage.bw",
                "bam": f"results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam",
                "bai": f"results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam.bai"
            })

        with open(output[0], 'w') as f:
            json.dump(tracks_config, f, indent=4)

```

Luego, simplemente modifiqué el json para que tenga el formato de configuración de IGV y lo inserté en mis templates:

```

rule generate_igv_report:
    input:
        bws = expand("results/mapping/alignments_STAR/{replic}/{replic}Coverage.bw", replic=REPLICS),
        bams = expand("results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam", replic=REPLICS),
        fasta = "data/reference/sacCer_Chrl.fa",
        gtf = "data/reference/sacCer_genes.gtf",
        template_base = "data/templates/igv_base.html",
        template_config = "data/templates/igv_config.js"
    output:
        html = "results/igv_visualization/igv_report.html",
        config = "results/igv_visualization/igv_config.js"
    run:
        import json
        import shutil
        import base64

        config = {
            "genome": {
                "id": "yeast_custom",
                "name": "Yeast Chromosome",
                "fastaURL": "data:base64," + base64.b64encode(open(input.fasta, "rb").read()).decode(),
                "tracks": [{
                    "name": "Annotations",
                    "type": "annotation",
                    "format": "gtf",
                    "url": "data:base64," + base64.b64encode(open(input.gtf, "rb").read()).decode(),
                    "displayMode": "EXPANDED"
                }]
            },
            "tracks": [
                {
                    "name": f"{replic} Coverage",
                    "type": "wig",
                    "format": "bigwig",
                    "url": "data:base64," + base64.b64encode(open(f"results/mapping/alignments_STAR/{replic}/{replic}Coverage.bw", "rb").read()).decode()
                }
                for replic in sorted(REPLICS)
            ]
        }

        with open(output.config, 'w') as f:
            f.write(f"const igvConfig = {json.dumps(config, indent=2)};")

        shutil.copy2(input.template_base, output.html)

        with open(output.html, 'a') as f_html:
            with open(output.config, 'r') as f_config:
                f_html.write(f"<script>\n{f_config.read()}\n</script>")

```

En este caso, pensando en que se genere de forma automática y auto-contenido tuve que codificar los bigwig con base64, a los fines de este trabajo no es tan relevante.

Una vez ejecutada la regla podemos ver el explorador del cromosoma con la cobertura y la anotación de forma interactiva en el navegador:



Conclusión

Finalmente integré todo en una regla en el Snakefile principal para poder correr todas estas reglas relevantes al alineamiento con un solo comando:

```
rule mapping_all:
    input:
        expand("results/mapping/alignments_STAR/{replic}/{replic}Aligned.sortedByCoord.out.bam", replic=REPLICS),
        expand("results/mapping/alignments_STAR/{replic}/{replic}ReadsPerGene.out.tab", replic=REPLICS),
        expand("results/mapping/alignments_STAR/{replic}/{replic}Coverage.bw", replic=REPLICS),
        expand("results/qc_qualimap/{replic}/qualimapReport.html", replic=REPLICS),
        "results/igv_visualization/igv_report.html"
```

Para correr esta regla luego:

```
snakemake --cores all mapping_all
```

Si visualizamos el grafo de ejecución de Snakemake, obtenemos lo siguiente:



De abajo para arriba, primero indexa los archivos necesarios, luego alinea para cada réplica. Luego indexa los bams y calcula la cobertura, con todas las coberturas genera el reporte de igv. Finalmente, en azul, se corren todos los control de calidad de qualimap y termina la regla `mapping_all`.

Otro punto que me parece importante y útil de utilizar Snakemake es que por default, guarda en `.snakemake/log` un registro de la ejecución de los flujos. Entonces podemos ver cómo se ejecutaron los comandos, en qué orden, cómo se definieron los parámetros, etc.

Expresión diferencial

Decidí usar el gtf que me dieron en la consigna porque es de una cepa en específico, es más preciso que usar un ensamblado de otra cepa.

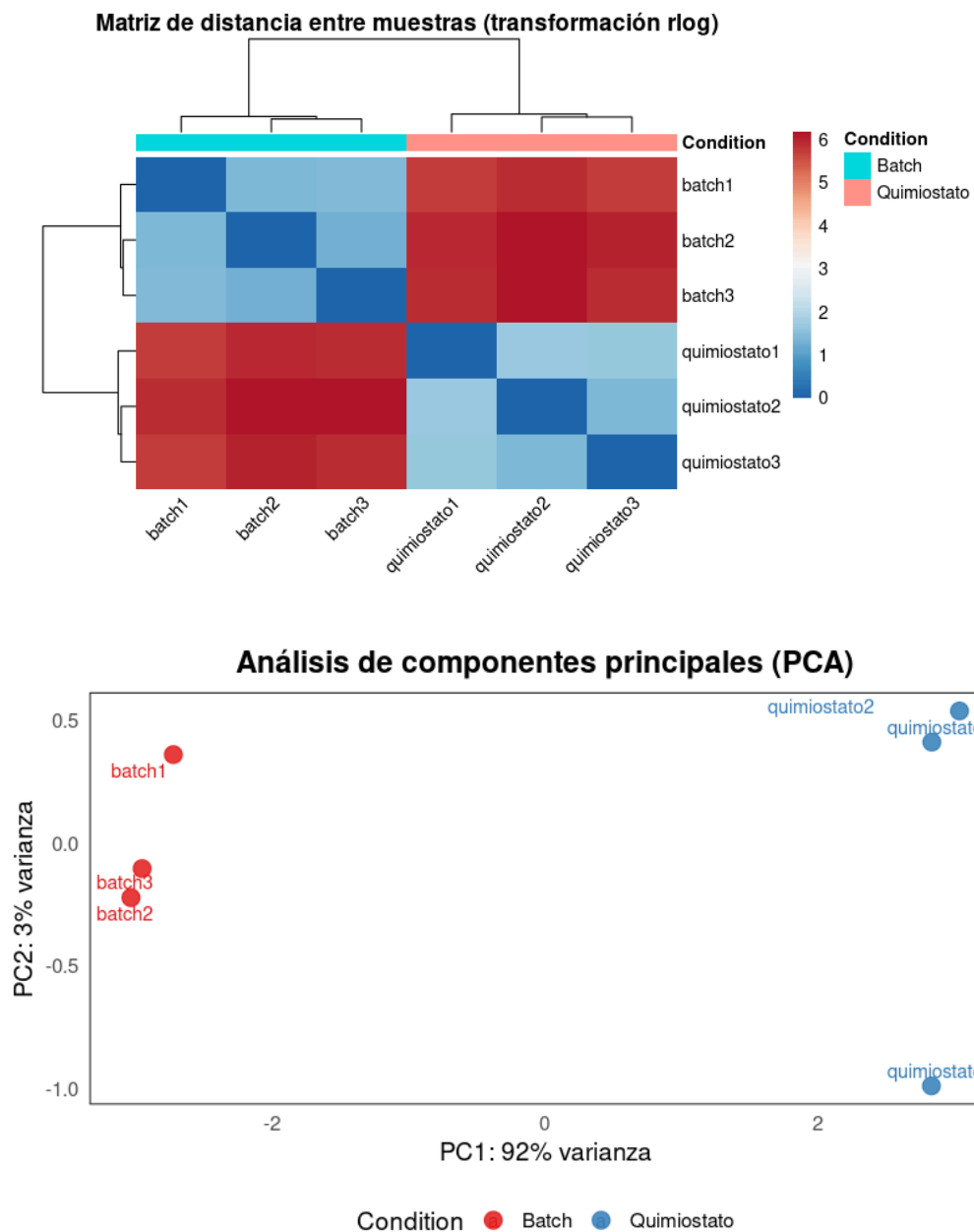
En mi GTF, en la columna no tengo type, entonces decido que sea exón o CDS. Decidí tomar exón en vez de CDS porque es más amplio. Con CDS solo estaría tomando genes que codifican para proteína, perdiendo por ejemplo ARNr o factores de transcripción.

Filtre genes de baja expresión, aquellos que no superaron 10 fueron descartados para reducir el ruido introducido. Con esto pasé de tener 127 genes a 100.

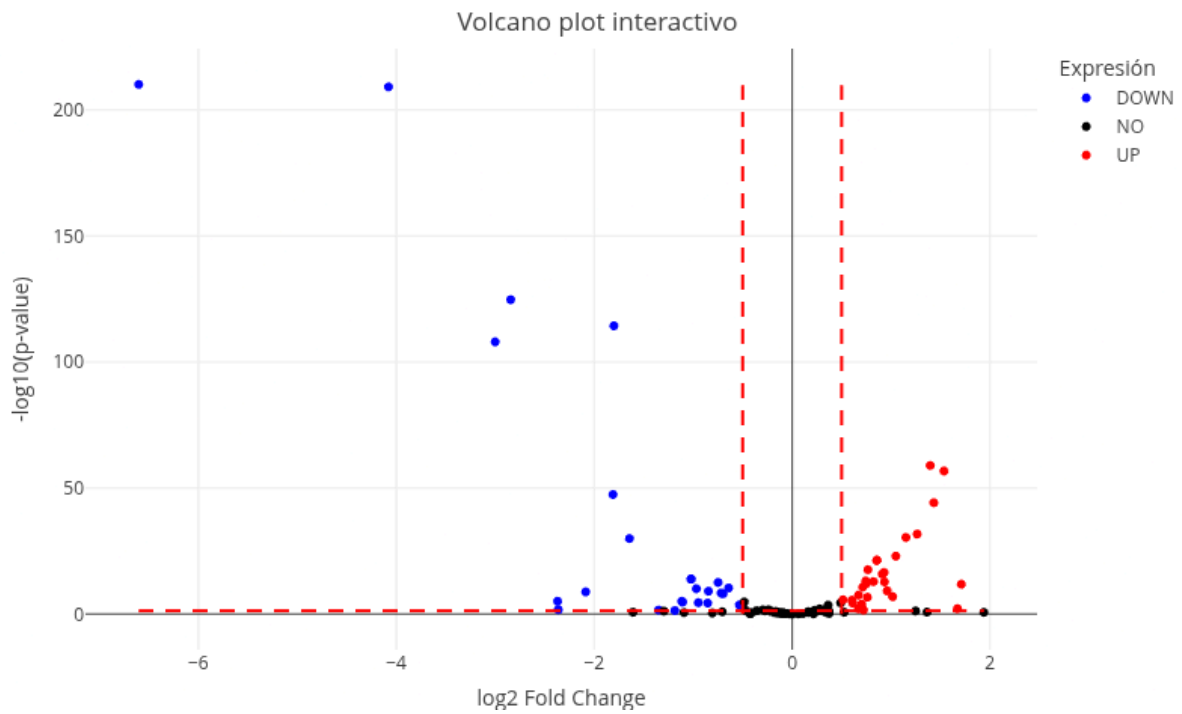
Luego normalice los datos y guardo los recuentos normalizados.

Finalmente realicé el análisis de expresión diferencial como Batch vs Quimiostato.

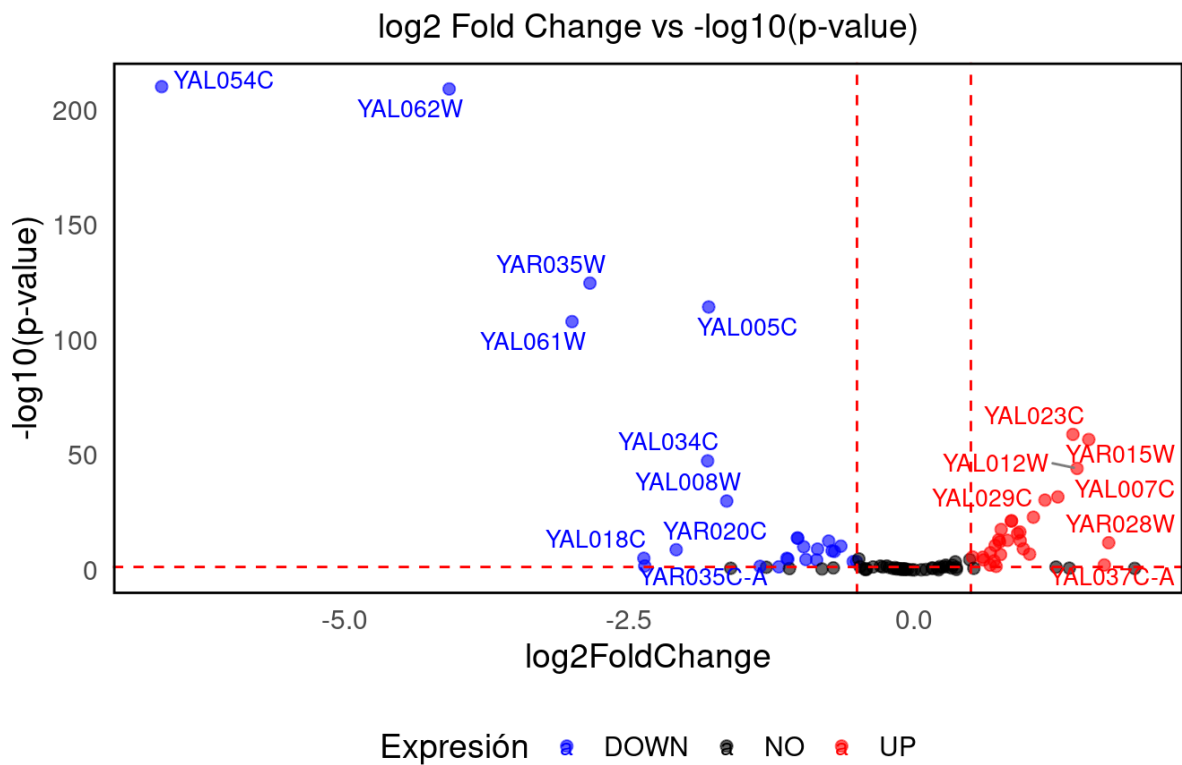
Luego realicé algunas visualizaciones de los datos, como la correlación de las muestras.



Después si pase a observar los volcano plot de los genes diferencialmente expresados con plotly y ggplot2.



Volcano Plot



La versión interactiva del volcano plot con Plotly se puede explorar en <https://justog220.github.io/tp-final-ngs/reports/differential-expression/index.html>

Interpretación de los resultados en el artículo.

Enriquecimiento funcional

Luego pasé a realizar el análisis funcional con YeastEnrichr, como me interesaba repetir este proceso con diversos datos, decidí automatizarlo. En la documentación del sitio web está como utilizar la API, yo debí adaptarlo a R. Cree funciones para enviar la lista de genes, obtener el token que devuelve y después bajar en forma de TSV los archivos para las librerías contra las que me gustaría contrastar. Con estas funciones desarrolladas me bajé los datos para:

- GO_Biological_Process_2018
- GO_Cellular_Component_2018
- GO_Molecular_Function_2018
- KEGG_2018

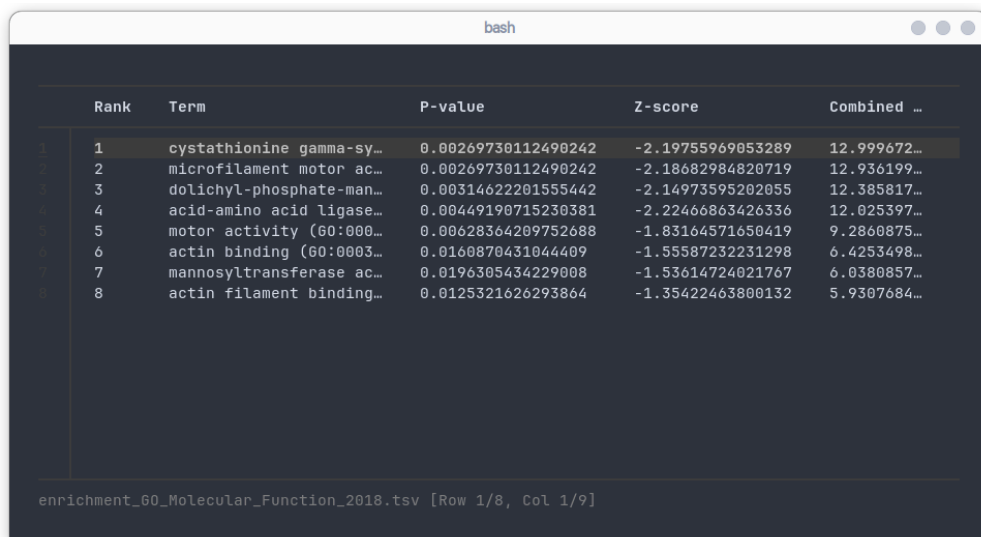
Para todas estas bibliotecas use una lista downregulados y otra upregulados.

Esta lista de genes las obtuve con los siguientes criterios:

- $p_{adj} < 0.01$
- $|\log_2(FC)| > 1$

Para hacer esto definí ciertas funciones que hacen lo que normalmente haríamos por la interfaz gráfica:

- `enviar_a_enrichr`: se le pasa como argumento una lista de genes, devuelve un token con el que se identifica este trabajo que estamos enviando al servidor.
- `run_enrichr`: se le pasa como argumento el identificador del trabajo y una librería de la que queremos obtener los resultados. También una ruta, en esta se guardan los resultados en formato TSV:



Rank	Term	P-value	Z-score	Combined ...
1	cystathionine gamma-sy...	0.00269730112490242	-2.19755969053289	12.999672...
2	microfilament motor ac...	0.00269730112490242	-2.18682984820719	12.936199...
3	dolichyl-phosphate-man...	0.00314622201555442	-2.14973595202055	12.385817...
4	acid-amino acid ligase...	0.00449190715230381	-2.22466863426336	12.025397...
5	motor activity (GO:000...	0.00628364209752688	-1.83164571650419	9.2860875...
6	actin binding (GO:0003...	0.0160870431044409	-1.55587232231298	6.4253498...
7	mannosyltransferase ac...	0.0196305434229008	-1.53614724021767	6.0380857...
8	actin filament binding...	0.0125321626293864	-1.35422463800132	5.9307684...

Después definí algunas otras funciones para cargar estos tsvs de forma más simple porque iba a ser un proceso muy repetitivo y graficar barplots y dotplots. Se puede visualizar esto en

<https://justog220.github.io/tp-final-ngs/reports/differential-expression/index.html#enriquecimiento-funcional-con-yeastenrichr> para no ensuciar este reporte.

Interpretación de los resultados en el artículo.

Links de interés

- Repositorio de github, con código snakekame y R utilizado:
<https://github.com/justog220/TP-Final-NGS>
- Página web con reportes HTML (MultiQCs, IGV y Análisis de expresión diferencial):
<https://justog220.github.io/tp-final-ngs/>